

Algoritmos Genéticos

Trabalho de desenvolvimento — Sessões 1 e 2
Resolução do caso-base, passo a passo

Filipe Azevedo · ISEP · Ano letivo 2026/2027

O que vamos fazer nestas duas sessões

- **Sessão 1 — A ferramenta**

- O que é um problema de otimização
- Como funciona um algoritmo genético
- A função `ga` do MATLAB, argumento a argumento
- Exemplo: mínimo e máximo da função seno

- **Sessão 2 — O problema**

- Custo de produção e custo incremental
- Despacho económico resolvido à mão, passo a passo
- O mesmo problema resolvido com a Toolbox
- Despacho com limite de potência num gerador

Não é preciso saber sistemas de energia

Tudo o que é necessário — barramento, gerador, carga, p.u., custo de produção — é introduzido de raiz na sessão 2. O que conta é o método de otimização.

Depois destas duas sessões

Cada grupo repete o mesmo raciocínio para o seu caso de estudo (tarefas T1 e T2), desenvolve o seu próprio algoritmo genético (T3) e constrói a APP (T4).

PARA COMEÇAR

```
d = dados_grupo(7);    % dados do seu grupo
help ga                % documentacao da funcao
```



1

A ferramenta

Sessão 1 · Otimização e algoritmos genéticos

Um problema de otimização tem três elementos

$$\text{minimizar } f(x) \quad \text{sujeito a } h(x) = 0 \quad \text{e} \quad lb \leq x \leq ub$$

1 Variáveis de decisão

O que podemos escolher. No nosso problema: as potências produzidas por cada gerador.

2 Função objetivo

O que queremos minimizar. No nosso problema: o custo total de produção, em €/h.

3 Restrições

O que a solução tem de cumprir: produzir exatamente o consumo e respeitar os limites das máquinas.

Não existe função de maximização

Para obter o máximo de $f(x)$, minimiza-se $-f(x)$ e troca-se o sinal do resultado. É exatamente o que vamos fazer com a função seno.

Como funciona um algoritmo genético

1 Gerar uma população inicial aleatória dentro do domínio

2 Avaliar o custo de cada indivíduo

3 Selecionar os melhores para reprodução (torneio)

4 Cruzar pares de progenitores

5 Mutar alguns descendentes

6 Guardar as elites e formar a nova geração

7 Repetir até ao critério de paragem

A ideia central

Em vez de seguir uma única solução, trabalha-se com uma POPULAÇÃO espalhada por todo o domínio. Não se usam derivadas: apenas se avalia a função.

O preço a pagar

É um método estocástico e aproximado: duas execuções dão resultados diferentes, e nenhuma garante o ótimo.

É este o ciclo da tarefa T3

Na tarefa T3 cada grupo implementa exatamente esta estrutura, sem usar a função ga.

Os quatro operadores

Seleção por torneio

Sorteiam-se k indivíduos; vence o de menor custo
 k pequeno $\rightarrow$ mais diversidade

Cruzamento BLX- α

$x_f = x_a + u (x_b - x_a)$, $u \in [-\alpha, 1+\alpha]$
 $\alpha \approx 0,25$

Mutação gaussiana

$x \leftarrow x + \sigma \cdot N(0,1)$
 σ decresce ao longo das gerações

Elitismo

Os E melhores passam intactos à geração seguinte
Sem ele, a melhor solução pode perder-se

Depois de cruzar e mutar, limitar sempre ao domínio

$x = \min(\max(x, lb), ub)$. Sem esta linha aparecem soluções impossíveis, como potências negativas. É o erro número um nas implementações da tarefa T3.

A função ga do MATLAB

```
[x, fval, exitflag, output] = ga(fun, nvars, A, b, Aeq, beq, lb, ub, nonlcon, options)
```

Argumento	Significado	No nosso problema
fun	Função a minimizar (recebe um vetor)	@(P) sum(d.b.*P + d.c.*P.^2, 2)
nvars	Número de variáveis	2 — as duas potências
A, b	Restrições lineares de desigualdade	Não usadas → []
Aeq, beq	Restrições lineares de igualdade	[1 1] e d.D → $P_1 + P_2 = D$
lb, ub	Limites de cada variável	d.Pmin e d.Pmax
nonlcon	Restrições não lineares	Não usadas → []
options	Opções do algoritmo	optimoptions(...)

Os argumentos que não se usam não se omitem

Substituem-se por [] mantendo a ordem. É por isso que aparecem quatro [] seguidos quando não há restrições lineares de desigualdade.

Exemplo: mínimo e máximo da função seno

Tarefa guiada G1

$$f(x) = \sin(x), \quad 0 \leq x \leq 2\pi$$

Porquê uma função que já sabemos resolver?

Precisamente por isso: serve para validar a utilização da ferramenta contra uma resposta conhecida.

	x obtido	valor exato	f(x)
Mínimo	4,7124	$3\pi/2 = 4,7124$	-1,0000
Máximo	1,5708	$\pi/2 = 1,5708$	+1,0000

A FUNÇÃO FITNESS

```
% Ficheiro seno.m
function z = seno(x)
z = sin(x);
```

FICHEIRO gaseno.m

```
options = optimoptions('ga', 'PlotFcn', ...
    {@gaplotbestf, @gaplotbestindiv, @gaplotexpectation, @gaplotstopping}, ...
    'InitialPopulationRange', [0; 2*pi]);

rng(1)
[xmin, fmin] = ga(@seno, 1, [], [], [], [], 0, 2*pi, [], options); % minimo
[xmax, fneg] = ga(@(x) -seno(x), 1, [], [], [], [], 0, 2*pi, [], options);
fmax = -fneg; % maximo
```

Três coisas a reter da sessão 1

1 A função fitness recebe um vetor

Mesmo com uma só variável, o argumento é $x(1)$. É a causa de metade dos erros iniciais.

2 O máximo obtém-se minimizando o simétrico

E o valor devolvido vem com o sinal trocado: $f_{\max} = -f_{\text{neg}}$.

3 O resultado muda de execução para execução

Use `rng(r)` para reproduzir. Nas tarefas do grupo são pedidas 10 execuções, com melhor valor, média e pior valor.



0 problema

Sessão 2 · Despacho económico de produção

Quatro noções, e chega

Barramento

Um nó da rede: o ponto onde se ligam geradores, cargas e linhas. É o equivalente a um nó de um circuito.

Gerador

Um grupo produtor térmico. A potência que produz é a nossa variável de decisão, entre um mínimo e um máximo técnicos.

Carga

O consumo ligado a um barramento. É um dado do problema: não se escolhe, tem de ser satisfeito.

Por unidade (p.u.)

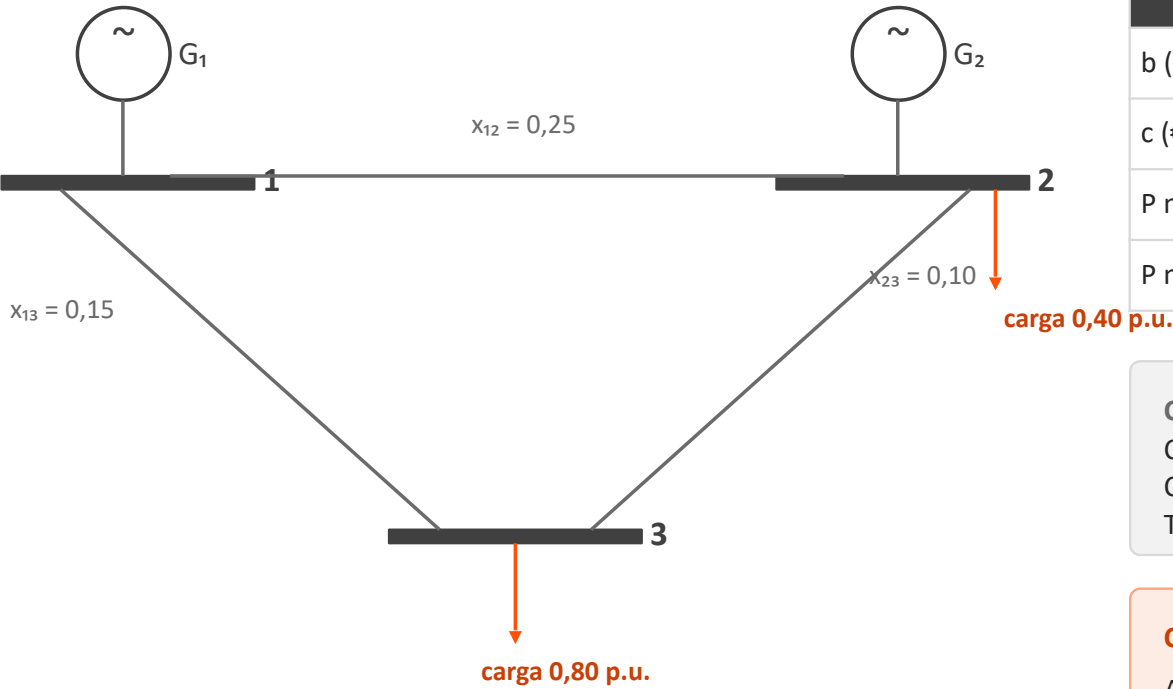
Normalização: valor em p.u. = valor real / valor de base. Com base 100 MVA, 1,00 p.u. = 100 MW.

Em todo o trabalho

As potências vêm em p.u. e os custos em €/h. Nunca é preciso converter para MW.

A rede em estudo

Caso-base: o mesmo para toda a turma



	G_1	G_2
$b \text{ (€/p.u.}\cdot\text{h)}$	0,30	0,20
$c \text{ (€/p.u.}^2\cdot\text{h)}$	0,12	0,30
P mínimo (p.u.)	0,10	0,10
P máximo (p.u.)	1,00	1,00

Carga total

Carga no barramento 2: 0,40 p.u.

Carga no barramento 3: 0,80 p.u.

Total: $D = 1,20 \text{ p.u.}$

Cada grupo tem outros números

A rede é a mesma para todos; mudam os coeficientes de custo, as cargas e o limite da tarefa T2. Ver o ANEXO I do enunciado.

Quanto custa produzir

$$C_1(P_1) = 0,30 \cdot P_1 + 0,12 \cdot P_1^2 \qquad C_2(P_2) = 0,20 \cdot P_2 + 0,30 \cdot P_2^2 \quad [\text{€/h}]$$

- O termo linear ($b \cdot P$) é o custo de combustível, aproximadamente proporcional à produção.
- O termo quadrático ($c \cdot P^2$) traduz a perda de rendimento: quanto mais perto do máximo, mais combustível por cada unidade adicional.
- Como $c > 0$, a função é convexa — o que garante que o mínimo encontrado é o mínimo global.

Repare bem nestes números

G_2 tem o combustível mais barato ($b = 0,20$ contra $0,30$), mas a curva mais inclinada ($c = 0,30$ contra $0,12$): encarece muito mais depressa com a produção.

Custo incremental (quanto custa produzir MAIS uma unidade)

$$dC_1/dP_1 = 0,30 + 0,24 \cdot P_1 \qquad dC_2/dP_2 = 0,20 + 0,60 \cdot P_2$$

Antes da matemática: o raciocínio

A pergunta certa

Suponha que G_1 custa 0,45 €/h por cada unidade adicional e G_2 custa 0,52. Compensa transferir produção de G_2 para G_1 ? Sim — poupa 0,07 por unidade transferida, sem alterar o total produzido.

Logo, no ótimo, as duas máquinas livres têm de ter o MESMO custo incremental

$$dC_1/dP_1 = dC_2/dP_2 = \lambda$$

- Se fossem diferentes, seria sempre possível baixar o custo total transferindo produção da mais cara para a mais barata — logo, não estaríamos no ótimo.
- A esse valor comum chama-se custo marginal do sistema, λ . Representa o custo de satisfazer mais uma unidade de carga.
- A dedução de Lagrange, nos próximos slides, confirma formalmente este raciocínio.

O problema, formulado

$$\begin{array}{ll} \text{minimizar} & C = 0,30 P_1 + 0,12 P_1^2 + 0,20 P_2 + 0,30 P_2^2 \\ \text{sujeito a} & P_1 + P_2 = 1,20 \quad (\text{balanço de potência}) \\ & 0,10 \leq P_i \leq 1,00 \quad (\text{limites técnicos}) \end{array}$$

1 Duas variáveis

P_1 e P_2 , as potências produzidas por cada gerador.

2 Uma igualdade

A produção total tem de igualar o consumo total: desprezamos as perdas.

3 Quatro desigualdades

Os limites mínimo e máximo de cada máquina.

Estratégia de resolução

Resolvemos primeiro ignorando os limites (desigualdades) e verificamos no fim se são cumpridos. Se forem, a solução está correta — é o que acontece neste caso.

Passo 1 — A função de Lagrange

- Não podemos simplesmente anular as derivadas do custo: as potências não são livres, têm de somar 1,20.
- A ideia de Lagrange: juntar objetivo e restrição numa só função, que já se pode derivar livremente.

$$\mathcal{L}(P_1, P_2, \lambda) = C_1(P_1) + C_2(P_2) + \lambda \cdot (D - P_1 - P_2)$$

1 O que é λ

Uma nova incógnita, o multiplicador de Lagrange. Passamos de 2 para 3 incógnitas, mas ganhamos uma equação: a própria restrição.

2 Porque funciona

Se a restrição for cumprida, o último termo é nulo e $\mathcal{L}$ coincide com o custo. Qualquer desvio ao balanço é penalizado por λ .

Passo 2 — Anular as derivadas

$$\partial \mathcal{L} / \partial P_i = dC_i / dP_i - \lambda = 0 \quad \Rightarrow \quad b_i + 2c_i P_i = \lambda$$

É o raciocínio de há dois slides, agora demonstrado

No ótimo, as duas máquinas têm o mesmo custo incremental, e esse valor comum é λ .

$$\text{Resolvendo em ordem a } P_i : \quad P_i = (\lambda - b_i) / (2c_i)$$

$$P_1 = (\lambda - 0,30) / 0,24$$

$$P_2 = (\lambda - 0,20) / 0,60$$

Passo 3 — Usar o balanço para obter λ

Substituímos P_1 e P_2 na equação de balanço $P_1 + P_2 = D$:

$$(\lambda - b_1)/(2c_1) + (\lambda - b_2)/(2c_2) = D$$

$$\lambda \cdot [1/(2c_1) + 1/(2c_2)] = D + b_1/(2c_1) + b_2/(2c_2)$$

$$\lambda = [D + b_1/(2c_1) + b_2/(2c_2)] / [1/(2c_1) + 1/(2c_2)]$$

Uma só incógnita, uma só equação

Depois de conhecer λ , as potências saem diretamente da expressão do passo 2. É todo o método.

Atenção: esta dedução assume que **NENHUM** limite está ativo.

Por isso, no fim, é obrigatório verificar se $0,10 \leq P_i \leq 1,00$.

Passo 4 — Cálculo de λ

Com $D = 1,20$ p.u.

Gerador	b_i	$2c_i$	$b_i/(2c_i)$	$1/(2c_i)$
G_1	0,30	0,24	1,2500	4,1667
G_2	0,20	0,60	0,3333	1,6667
Soma			1,5833	5,8333

$$\lambda = (1,20 + 1,5833) / 5,8333$$

$$\lambda = 2,7833 / 5,8333$$

$$\lambda = 0,4771 \text{ €/p.u.}\cdot\text{h}$$

$$P_1 = (0,4771 - 0,30) / 0,24 = 0,7381 \text{ p.u.}$$

$$P_2 = (0,4771 - 0,20) / 0,60 = 0,4619 \text{ p.u.}$$

Já temos a solução — falta provar que está certa

Três verificações: o balanço, os limites e a igualdade dos custos incrementais.

Passo 5 — As três verificações

1 Balanço

$$0,7381 + 0,4619 = 1,2000$$

Igual a D. ✓

2 Limites

$$0,10 \leq 0,7381 \leq 1,00$$

$$0,10 \leq 0,4619 \leq 1,00$$

A hipótese confirma-se. ✓

3 Custos incrementais

$$0,30 + 0,24 \times 0,7381 = 0,4771$$

$$0,20 + 0,60 \times 0,4619 = 0,4771$$

Iguais, e iguais a λ . ✓

Custo total

$$C_1 = 0,30 \times 0,7381 + 0,12 \times 0,7381^2 = 0,2214 + 0,0654 = 0,2868 \text{ €/h}$$

$$C_2 = 0,20 \times 0,4619 + 0,30 \times 0,4619^2 = 0,0924 + 0,0640 = 0,1564 \text{ €/h}$$

$$C = 0,4432 \text{ €/h}$$

A terceira verificação é a mais importante

Se os custos incrementais não derem iguais, há erro de cálculo — mesmo que o balanço e os limites estejam corretos.

O resultado é contra-intuitivo — e é esse o ponto

	G ₁	G ₂
Combustível (b)	0,30 — mais caro	0,20 — mais barato
Inclinação (c)	0,12 — suave	0,30 — acentuada
Produção obtida	0,7381 p.u.	0,4619 p.u.

Porque produz mais a máquina do combustível mais caro?

Porque o que decide não é b, mas o custo incremental $b + 2cP$. G_2 começa mais barata, mas encarece três vezes mais depressa; a partir de certo ponto deixa de compensar.

No ponto ótimo: $0,30 + 0,24 \times 0,7381 = 0,20 + 0,60 \times 0,4619 = 0,4771$

A conclusão a levar para o relatório

O despacho económico não é "carregar primeiro a máquina mais barata". É igualar os custos incrementais — e o resultado depende tanto de b como de c.

O mesmo problema, agora com a Toolbox

DESPACHO COM A FUNÇÃO `ga`

```
d = dados_grupo(0); % dados do caso-base

fcusto = @(P) sum(d.b.*P + d.c.*P.^2, 2);

opts = optimoptions('ga', 'PopulationSize', 50, ...
    'MaxGenerations', 100);

rng(1)
[P, C] = ga(fcusto, 2, [], [], [1 1], d.D, ...
    d.Pmin, d.Pmax, [], opts)

% verificacao: devem dar dois valores iguais
lambda = d.b + 2*d.c.*P
```

1 O balanço

$A_{eq} = [1 \ 1]$ e $b_{eq} = d.D$ impõem $P_1 + P_2 = 1,20$. A função `ga` só gera soluções que cumprem esta igualdade.

2 Onde está λ ?

A função `ga` não o devolve. Calcula-se depois, pelo custo incremental de qualquer máquina que não esteja num limite.

	Lagrange	Algoritmo genético
Solução	Exata: 0,7381 / 0,4619	Aproximada; varia ligeiramente entre execuções
Aplicabilidade	Só com custos convexos e deriváveis	Qualquer função, mesmo sem derivada

Estudo estatístico: uma execução não é um resultado

ESTRUTURA A USAR EM TODAS AS TAREFAS

```
nexec = 10; R = zeros(nexec,1);  
for r = 1:nexec  
    rng(r) % semente diferente  
    [~, R(r)] = ga(fcusto, 2, [], [], [1 1], d.D, ...  
                  d.Pmin, d.Pmax, [], opts);  
end  
fprintf('melhor %.4f | media %.4f | pior %.4f\n', ...  
        min(R), mean(R), max(R));
```

Porquê 10 execuções?

Porque o algoritmo é estocástico. Uma execução isolada não permite concluir nada sobre o desempenho do método.

O que esperar neste problema

As 10 execuções devem dar praticamente o mesmo valor (diferenças na 4.^a casa decimal). O problema é convexo e tem duas variáveis: é fácil. Diga-o no relatório — não apresente a coincidência como mérito do algoritmo.

E se der muito diferente?

Verifique se a restrição de balanço está a ser imposta por Aeq/beq e se os limites lb e ub estão corretos. É quase sempre aí que está o erro.

E se um gerador ficar limitado?

Tarefa T2

A situação

Por indisponibilidade parcial, G_1 passa a poder produzir no máximo 0,60 p.u. A solução anterior (0,7381) deixa de ser admissível.

1 Detetar a violação

Resolver como em T1 e verificar os limites: $0,7381 > 0,60$. ✗

2 Fixar no limite

A máquina que viola o limite fica encostada a ele: $P_1 = 0,60$ p.u.

3 Balanço dá a outra

$P_2 = D - P_1 = 1,20 - 0,60 = 0,60$ p.u.

4 Recalcular λ

λ passa a ser o custo incremental da única máquina livre, G_2 .

$$\lambda = 0,20 + 0,60 \times 0,60 = 0,5600 \text{ €/p.u.}\cdot\text{h}$$

A verificação que não pode faltar

Custo incremental de G_1 no limite:

$$0,30 + 0,24 \times 0,60 = 0,4440 < \lambda = 0,5600$$

Como se lê este resultado

G_1 continua a ser a máquina mais barata na margem: produziria mais se pudesse. Está por isso corretamente encostada ao limite superior. Se a desigualdade saísse ao contrário, o limite não deveria estar ativo — e haveria erro no procedimento.

	P_1 (p.u.)	P_2 (p.u.)	λ	Custo (€/h)
T1 — sem limitação	0,7381	0,4619	0,4771	0,4432
T2 — com $G_1 \leq 0,60$	0,6000	0,6000	0,5600	0,4512
Variação	-0,1381	+0,1381	+0,0829	+0,0080

O custo da avaria

+0,0080 €/h, cerca de 1,8 %. E λ sobe: com a máquina eficiente limitada, satisfazer mais carga fica mais caro.

Na Toolbox, muda uma única coisa

DESPACHO COM LIMITE

```
Pm = d.Pmax; Pm(1) = 0.60; % novo limite maximo de G1

rng(1)
[P2, C2] = ga(fcusto, 2, [], [], [1 1], d.D, d.Pmin, Pm, [], opts)
%          ^^^^
%          so muda o argumento ub
```

O que NÃO muda

A função objetivo, o número de variáveis, a restrição de balanço e os limites mínimos. Tudo isso se mantém.

O que o resultado deve dar

$P = [0,6000 \ 0,6000]$ e $C = 0,4512$ €/h — exatamente o valor calculado à mão.

Cuidado frequente

Alterar o limite na tabela de dados e esquecer de o passar à função ga. A APP tem de ler sempre os dados da interface, nunca valores fixos no código.

Preparação da tarefa T3: uma única variável

- Com dois geradores e a restrição $P_1 + P_2 = D$, há apenas UM grau de liberdade.
- Se o vosso algoritmo gerar só P_1 e calcular $P_2 = D - P_1$, o balanço fica satisfeito por construção — sem penalidades nem correções.

$$C(P_1) = b_1 P_1 + c_1 P_1^2 + b_2 (D - P_1) + c_2 (D - P_1)^2$$

Mas atenção ao domínio de P_1 : também P_2 tem de respeitar os seus limites

$$\max(P_1^{\min}, D - P_2^{\max}) \leq P_1 \leq \min(P_1^{\max}, D - P_2^{\min})$$

No caso-base: $\max(0,10 ; 0,20) = 0,20 \leq P_1 \leq \min(1,00 ; 1,10) = 1,00$

É aqui que a maioria vai errar

Gerar P_1 entre 0,10 e 1,00 produz soluções em que P_2 ultrapassa os seus limites — e o resultado sai errado sem dar erro.

Resumo: o método em sete passos

1 Escrever a função de Lagrange

$$\mathcal{L} = C_1 + C_2 + \lambda (D - P_1 - P_2)$$

2 Anular as derivadas

$$b_i + 2c_i P_i = \lambda \implies P_i = (\lambda - b_i)/(2c_i)$$

3 Substituir no balanço

$$\lambda = [D + \sum b_i/(2c_i)] / \sum 1/(2c_i)$$

4 Calcular λ e as potências

$$\lambda = 0,4771 ; P = (0,7381 ; 0,4619)$$

5 Verificar balanço, limites e custos incrementais

Os três têm de bater certo

6 Calcular o custo total

$$C = 0,4432 \text{ €/h}$$

7 Se um limite for violado, fixar a máquina e repetir

$$\text{Tarefa T2: } \lambda = 0,5600 ; C = 0,4512 \text{ €/h}$$

O que fazer a seguir

1 Tarefa T1

Repetir todo este raciocínio para o caso de estudo do vosso grupo, pelos dois métodos, com estudo estatístico de 10 execuções.

2 Tarefa T2

Aplicar o procedimento de fixação ao gerador e ao limite indicados no vosso caso.

3 Tarefas T3 e T4

Desenvolver o algoritmo genético próprio e construir a APP, a partir do ANEXO II do enunciado.

Comecem já pela APP, em paralelo

Não deixem a aplicação para o fim. Construam um separador que resolva a tarefa T1 e acrescentem as restantes à medida que avançam.

Na apresentação

O docente vai alterar um dado na vossa APP e pedir a interpretação do novo resultado. Serão colocadas questões individuais a cada elemento do grupo.

Bom trabalho

Dúvidas nas aulas PL ou por correio eletrónico — fta@isep.ipp.pt

